

Memoria Práctica 2

Grupo 1201: Antonio Morono y Pedro Ismael Haddou

Acceso a la base de datos

Tras leer la documentación adjuntada en el enunciado comprendimos en cierto modo el funcionamiento de la librería `odbc` y como se implementaba en C. El tutorial proporcionado se basa en ejemplos y supone que se cuenta con una buena referencia de la API de ODBC. Se especifican requisitos previos, como un compilador ANSI C, un administrador de controladores ODBC, y un controlador ODBC y base de datos operativos. Se recomienda el uso del administrador de controladores `unixODBC`.

Se detalla la importancia de los manejadores en ODBC, como `SQLHENV` para el entorno, `SQLHDBC` para la conexión, `SQLHSTMT` para las consultas SQL, y `SQLHDESC` para descriptores, con la necesidad de asignar y liberar estos manejadores en un orden específico. Además, se explica cómo configurar el entorno, especificar el comportamiento deseado de ODBC y listar controladores y fuentes de datos instalados tanto de manera manual como programática.

Programa a implementar

Modificamos el menú dado para que se adapte a las consultas solicitadas (añadiendo el `date`) e implementamos la paginación que permite crear una segunda página a la que se puede acceder con las teclas "Re Pág/Av Pág" en el caso de obtener un resultado con un número de filas que sobrepasa el menú. Además para las consultas tuvimos que crear un nuevo campo en la estructura `MENU` y modificar las funciones dadas para que al presionar `enter` sobre uno de los resultados se muestre información adicional.

Consultas a realizar

Realización de búsquedas

La implementamos con la siguiente query:

```
--query search
--Calculamos los asientos disponibles para cada vuelo en un rango de fechas
WITH asientos_totales AS (
    SELECT
        f.flight_id,
        COUNT(s.seat_no) AS asientos_totales,
        f.scheduled_departure
    FROM
        flights f
        JOIN aircrafts_data ad ON ad.aircraft_code = f.aircraft_code
        JOIN seats s ON s.aircraft_code = ad.aircraft_code
    WHERE
        DATE(f.scheduled_departure) >= ?
        AND DATE(f.scheduled_departure) <= ?
    GROUP BY
        f.flight_id
```

```

),
asientos_reservados AS (
    SELECT
        f.flight_id,
        COUNT(tf.ticket_no) AS asientos_reservados
    FROM
        ticket_flights tf
        JOIN flights f ON tf.flight_id = f.flight_id
    WHERE
        DATE(f.scheduled_departure) >= ?
        AND DATE(f.scheduled_departure) <= ?
    GROUP BY
        f.flight_id
),
asientos_vacios AS (
    SELECT
        AT.flight_id,
        (AT.asientos_totales - COALESCE(ar.asientos_reservados,0)) AS
vacios
    FROM
        asientos_totales AT
        LEFT JOIN asientos_reservados ar ON ar.flight_id = AT.flight_id
),
--Calculamos los vuelos salientes del aeropuerto de salida
salidas AS (
    SELECT
        f.flight_id,
        f.departure_airport,
        f.arrival_airport,
        f.scheduled_departure,
        f.scheduled_arrival,
        AT.vacios AS asientos,
        f.aircraft_code
    FROM
        flights f
        JOIN asientos_vacios AT ON f.flight_id = AT.flight_id
    WHERE
        f.departure_airport = ?
        AND AT.vacios <> 0
        AND DATE(f.scheduled_departure)=?
),
--Calculamos los vuelos salientes del aeropuerto de salida
llegadas AS (
    SELECT
        f.flight_id,
        f.departure_airport,
        f.arrival_airport,
        f.scheduled_departure,
        f.scheduled_arrival,
        AT.vacios AS asientos,
        f.aircraft_code
    FROM
        flights f
        JOIN asientos_vacios AT ON f.flight_id = AT.flight_id

```

```

WHERE
    f.arrival_airport = ?
    AND AT.vacios <> 0
)
--Calculamos vuelos directos y con transbordo y unimos
SELECT
    first_flight_id,
    fecha_salida_1,
    fecha_llegada_2,
    transbordos,
    asientos_libres,
    aircraft_code_1,
    second_flight_id,
    fecha_salida_2,
    fecha_llegada_1,
    aircraft_code_2
FROM
    (
        --Calculamos los vuelos con transbordo
        SELECT
            s.flight_id AS first_flight_id,
            s.scheduled_departure AS fecha_salida_1,
            s.scheduled_arrival AS fecha_llegada_2,
            s.departure_airport,
            l.arrival_airport,
            '1' AS transbordos,
            CASE WHEN s.asientos >= l.asientos THEN l.asientos ELSE
s.asientos END AS asientos_libres,
            s.aircraft_code AS aircraft_code_1,
            l.flight_id AS second_flight_id,
            l.scheduled_departure AS fecha_salida_2,
            l.scheduled_arrival AS fecha_llegada_1,
            l.aircraft_code AS aircraft_code_2
        FROM
            salidas s
            JOIN llegadas l ON s.arrival_airport = l.departure_airport
        WHERE
            s.scheduled_arrival < l.scheduled_departure
            AND (l.scheduled_arrival - s.scheduled_departure) <=
'24:00:00.000'
        UNION
        --Calculamos los vuelos directos
        SELECT
            s.flight_id AS first_flight_id,
            s.scheduled_departure AS fecha_salida_1,
            s.scheduled_arrival AS fecha_llegada_2,
            s.departure_airport,
            s.arrival_airport,
            '0' AS transbordos,
            s.asientos,
            s.aircraft_code AS aircraft_code_1,
            s.flight_id AS second_flight_id,
            s.scheduled_departure AS fecha_salida_2,
            s.scheduled_arrival AS fecha_llegada_1,

```

```
        s.aircraft_code AS aircraft_code_2
    FROM
        salidas s
    WHERE
        s.arrival_airport = ?

) AS todos
ORDER BY
    (fecha_llegada_1 - fecha_salida_1) ASC;
```

Esta query SQL realiza lo siguiente:

1. Common Table Expressions (CTE):

- **asientos_totales**: Calcula el número total de asientos disponibles para cada vuelo.
- **asientos_reservados**: Calcula el número de asientos reservados para cada vuelo.
- **asientos_vacios**: Calcula la diferencia entre los asientos totales y los asientos reservados para obtener los asientos vacíos.

2. Subconsultas de vuelos con asientos vacíos:

- **salidas y llegadas**: Filtran los vuelos de salida y llegada que cumplen ciertas condiciones, como aeropuertos específicos y asientos vacíos no nulos.

3. Consulta principal:

- Combina las subconsultas **salidas** y **llegadas** para obtener los vuelos que cumplen con ciertas condiciones de transbordo.
- La consulta utiliza **UNION** para combinar los resultados de vuelos con transbordo y vuelos directos.

4. Selección y ordenación final:

- La consulta principal selecciona y organiza los resultados finales para mostrar la información relevante, como las fechas de salida y llegada, el número de transbordos, los asientos disponibles y los códigos de aeronaves.

Una vez desarrollada la query y comprobado su correcto funcionamiento la implementamos en C haciendo uso de la librería **odbc** de la siguiente manera:

1. **Archivos de cabecera y prototipo de función**: El código comienza incluyendo los ficheros de cabecera necesarios como **string.h**, **stdlib.h**, **stdio.h**, **sql.h**, **sqlext.h**, y dos ficheros de cabecera personalizados: **odbc.h** y **search.h**. A continuación, declara la función **results_search**, diseñada para interactuar con una base de datos y devolver los resultados de la búsqueda de vuelos.
2. **Declaración de variables**: La función comienza declarando varias variables que se utilizarán a lo largo de la función. Entre ellas se incluyen los manejadores de entorno, conexión y sentencia para ODBC, el estado de retorno SQL y varias estructuras de datos SQL para manejar datos de fecha y hora. También declara varias matrices de caracteres para almacenar los resultados de la consulta SQL.

3. **Conectar a la base de datos:** La función llama a la función `odbc_connect` para establecer una conexión con la base de datos. Esta función no está definida en el código proporcionado, pero presumiblemente configura el entorno ODBC y la conexión utilizando las funciones ODBC `SQLAllocHandle` y `SQLConnect`.
4. **Preparar sentencia SQL:** La función prepara una sentencia SQL para su ejecución utilizando la función `SQLPrepare`. Esta sentencia SQL es una consulta compleja que obtiene datos de vuelo de la base de datos.
5. **Agregar Parámetros a la Sentencia SQL:** La función vincula parámetros a la sentencia SQL utilizando la función `SQLBindParameter`. Estos parámetros corresponden a los marcadores de posición de la sentencia SQL y se utilizan para filtrar los datos que devuelve la consulta.
6. **Ejecutar sentencia SQL:** La función ejecuta la sentencia SQL utilizando la función `SQLExecute`. Esto envía la sentencia SQL a la base de datos para su ejecución.
7. **Agregar Columnas al Conjunto de Resultados:** La función vincula las columnas del conjunto de resultados a las variables del programa utilizando la función `SQLBindCol`. Esto permite al programa acceder a los datos devueltos por la consulta SQL.
8. **Obtener resultados y procesar datos:** La función obtiene los resultados de la consulta SQL utilizando la función `SQLFetch` en un bucle. Para cada fila del conjunto de resultados, construye dos cadenas que contienen los datos del vuelo y las copia en las matrices `choices` y `message`. El número de filas obtenidas se almacena en `n_choices`.
9. **Cerrar el cursor y liberar recursos:** Una vez obtenidos todos los resultados, la función cierra el cursor utilizando la función `SQLCloseCursor` y libera el manejador de la sentencia utilizando la función `SQLFreeHandle`. Luego se desconecta de la base de datos usando la función `odbc_disconnect`.

Nuestro código permite el uso de la API ODBC para interactuar con la base de datos **flight**. Muestra cómo preparar y ejecutar consultas SQL, vincular parámetros y columnas a estas consultas, y obtener y procesar los resultados.

Nota: Un aspecto importante de la query es que para los vuelos con transbordo devuelve toda la información (la que aparece en la pantalla y la del mensaje) y toda esta información se lleva a las variables correspondientes en C. Sin embargo, para los vuelos sin transbordo imprime ciertos campos duplicados para evitar el uso de NULL que ha sido causa de errores. Entonces no toda esa información se lleva a las variables de C, solo la necesaria.

Emisión de tarjetas de embarque

La implementamos con 2 consultas. A continuación se muestra la primera consulta:

```
--query1 bpass
--Encontramos asientos para el vuelo correspondiente
--a la reserva
WITH vuelos AS
  (SELECT tf.flight_id,
          t.ticket_no,
```

```

        ROW_NUMBER() OVER (PARTITION BY tf.flight_id
                           ORDER BY t.ticket_no) AS posicion
FROM tickets t
INNER JOIN ticket_flights tf ON t.ticket_no = tf.ticket_no
WHERE t.book_ref = ?
    AND tf.flight_id NOT in
        (SELECT bp.flight_id
         FROM boarding_passes bp
         WHERE bp.ticket_no = t.ticket_no)
GROUP BY tf.flight_id,
         t.ticket_no
ORDER BY tf.flight_id,
         t.ticket_no),
asientos AS
(SELECT DISTINCT f.flight_id,
                s.seat_no
 FROM flights f
 INNER JOIN seats s ON f.aircraft_code = s.aircraft_code
 INNER JOIN vuelos v ON f.flight_id = v.flight_id),
asientos_ocupados AS
(SELECT DISTINCT bp.flight_id,
                bp.seat_no
 FROM boarding_passes bp
 INNER JOIN vuelos v ON bp.flight_id = v.flight_id),
asientos_vacios AS
(SELECT DISTINCT a.flight_id,
                a.seat_no,
                row_number() OVER (PARTITION BY a.flight_id
                                   ORDER BY a.flight_id,
                                           a.seat_no) AS numero
 FROM asientos a
 WHERE a.flight_id not in
        (SELECT ao.flight_id
         FROM asientos_ocupados ao
         WHERE ao.seat_no = a.seat_no ) )
SELECT t.passenger_name,
       v.flight_id,
       f.scheduled_departure,
       av.seat_no
FROM asientos_vacios av,
     vuelos v,
     tickets t,
     flights f
WHERE v.flight_id = av.flight_id
    AND v.posicion = av.numero
    AND t.ticket_no = v.ticket_no
    AND f.flight_id = v.flight_id;

```

Esta query SQL realiza lo siguiente:

1. Common Table Expressions (CTE):

- **vuelos**: Esta CTE selecciona los vuelos correspondientes a la reserva y asigna un número de posición a cada vuelo en función del número de reserva.
- **asientos**: Esta CTE selecciona los asientos disponibles para cada vuelo.
- **asientos_ocupados**: Esta CTE selecciona los asientos que ya están ocupados para cada vuelo.
- **asientos_vacios**: Esta CTE selecciona los asientos vacíos para cada vuelo.

2. Consulta principal:

- La consulta principal une las CTEs **asientos_vacios**, **vuelos**, **tickets** y **flights** para obtener los detalles de los asientos vacíos para cada vuelo de la reserva.
- La consulta utiliza la cláusula **WHERE** para filtrar los resultados basándose en el ID del vuelo, el número de posición y el número de reserva.

A continuación se muestra la segunda consulta:

```
--query2 bpass
--Asignamos los asientos a la
WITH vuelos AS
  (SELECT tf.flight_id,
         t.ticket_no,
         ROW_NUMBER() OVER (PARTITION BY tf.flight_id
                           ORDER BY t.ticket_no) AS posicion
   FROM tickets t
   INNER JOIN ticket_flights tf ON t.ticket_no = tf.ticket_no
   WHERE t.book_ref = ?
     AND tf.flight_id NOT IN
     (SELECT bp.flight_id
      FROM boarding_passes bp
      WHERE bp.ticket_no = t.ticket_no)
   GROUP BY tf.flight_id,
            t.ticket_no
   ORDER BY tf.flight_id,
            t.ticket_no),
  asientos AS
  (SELECT DISTINCT f.flight_id,
                  s.seat_no
   FROM flights f
   INNER JOIN seats s ON f.aircraft_code = s.aircraft_code
   INNER JOIN vuelos v ON f.flight_id = v.flight_id),
  asientos_ocupados AS
  (SELECT DISTINCT bp.flight_id,
                  bp.seat_no
   FROM boarding_passes bp
   INNER JOIN vuelos v ON bp.flight_id = v.flight_id),
  asientos_vacios AS
  (SELECT DISTINCT a.flight_id,
                  a.seat_no,
                  row_number() OVER (PARTITION BY a.flight_id
                                    ORDER BY a.flight_id,
                                             a.seat_no) AS numero
   FROM asientos a
```

```

WHERE a.flight_id not in
      (SELECT ao.flight_id
       FROM asientos_ocupados ao
       WHERE ao.seat_no = a.seat_no ) )
INSERT INTO boarding_passes (ticket_no, flight_id, boarding_no,
seat_no)
SELECT t.ticket_no,
       v.flight_id,
       COALESCE(
         (SELECT MAX(bp.boarding_no)
          FROM boarding_passes bp
          WHERE bp.flight_id = v.flight_id), 0) + ROW_NUMBER()

OVER (

ORDER BY av.seat_no) , av.seat_no
FROM asientos_vacios av,
     vuelos v,
     tickets t,
     flights f,
     boarding_passes bp
WHERE v.flight_id = av.flight_id
     AND v.posicion = av.numero
     AND t.ticket_no = v.ticket_no
     AND f.flight_id = v.flight_id
GROUP BY t.ticket_no,
         v.flight_id,
         av.seat_no;

```

Esta query SQL realiza lo siguiente:

1. Common Table Expressions (CTE):

- **vuelos**: Esta CTE selecciona los vuelos correspondientes a la reserva y asigna un número de posición a cada vuelo en función del número de boleto.
- **asientos**: Esta CTE selecciona los asientos disponibles para cada vuelo.
- **asientos_ocupados**: Esta CTE selecciona los asientos que ya están ocupados para cada vuelo.
- **asientos_vacios**: Esta CTE selecciona los asientos vacíos para cada vuelo.

2. Consulta principal:

- La consulta principal une las CTEs **asientos_vacios**, **vuelos**, **tickets** y **flights** para obtener los detalles de los asientos vacíos para cada vuelo de la reserva.
- La consulta utiliza la cláusula **WHERE** para filtrar los resultados basándose en el ID del vuelo, el número de posición y el número de reserva.
- La consulta utiliza la sentencia **INSERT INTO** para insertar los detalles de los asientos vacíos en la tabla **boarding_passes**.

Una vez desarrolladas las consultas y verificado su correcto funcionamiento, las implementamos en C utilizando la biblioteca **odbc**. La implementación es similar a la que realizamos anteriormente para las búsquedas. Las diferencias radican en la ejecución de dos consultas: se repiten los puntos del 4 al 8 explicados en la implementación de las búsquedas. En cuanto a las variables, las ajustamos a los atributos de las tablas que se mostrarán en el menú.

Nota: Un aspecto crucial de la consulta es que, para reservas que abarcan múltiples pasajeros, se asignan tarjetas de embarque individuales a cada persona. Además, los asientos asignados a cada pasajero siguen estando juntos, manteniendo así la proximidad durante el vuelo.